

BookingHub: Sistema de Reservas de Voos e Hotéis Utilizando PostgreSQL e FastAPI

José Ivan Luz Ramos, Pedro Rhuan de Oliveira Sales
Jonas Miguel de Oliveira Costa, Carlos Neuan Sampaio Silva

¹ Centro Universitário Católica de Quixadá – Unicatólica
Curso de Sistemas de Informação
Disciplina: Implementação em Banco de Dados
Quixadá – Ceará – Brasil

Abstract. *This work presents the development of the BookingHub platform, a flight and hotel reservation system implemented using PostgreSQL and Python through the FastAPI framework. The objective of the system is to support environments with a large number of simultaneous accesses while ensuring performance, consistency, and data availability. Fundamental database concepts were applied during the project development, including query processing, index creation and usage, execution plan analysis, transaction and concurrency control, as well as failure recovery mechanisms provided by PostgreSQL. Performance tests were also conducted using SQL queries and execution plan analysis through the EXPLAIN (ANALYZE, BUFFERS) command, enabling the evaluation of the impact of indexes on query performance.*

Keywords: *PostgreSQL, Database, Indexes, Concurrency, Transactions, Failure Recovery.*

Resumo. *Este trabalho apresenta o desenvolvimento da plataforma BookingHub, um sistema de reservas de voos e hotéis implementado utilizando PostgreSQL e Python por meio do framework FastAPI. O objetivo do sistema é suportar ambientes com grande quantidade de acessos simultâneos, garantindo desempenho, consistência e disponibilidade dos dados. Durante o desenvolvimento do projeto foram aplicados conceitos fundamentais de banco de dados, incluindo processamento de consultas, criação e utilização de índices, análise de planos de execução, controle de transações e concorrência, além de mecanismos de recuperação de falhas fornecidos pelo PostgreSQL. Também foram realizados testes de desempenho utilizando consultas SQL e análise dos planos de execução por meio do comando EXPLAIN (ANALYZE, BUFFERS), permitindo avaliar o impacto dos índices no desempenho das consultas.*

Palavras-chave: *PostgreSQL, Banco de Dados, Índices, Concorrência, Transações, Recuperação de Falhas.*

1. Introdução

A plataforma BookingHub consiste em um sistema de reservas online destinado ao gerenciamento de passagens aéreas e reservas de hotéis. Em ambientes com grande quantidade de usuários simultâneos, torna-se necessário que o banco de dados possua mecanismos capazes de manter consistência, desempenho e disponibilidade. Neste trabalho foi desenvolvido um banco de dados utilizando PostgreSQL para suportar as funcionalidades da plataforma, contendo estruturas relacionais para armazenamento das informações

de aeroportos, hotéis, voos, quartos, clientes, reservas e pagamentos. Também foram aplicados conceitos relacionados ao processamento de consultas, controle de concorrência e recuperação de falhas. O banco de dados PostgreSQL foi utilizado como principal sistema gerenciador devido aos seus mecanismos de desempenho, integridade e recuperação de falhas [PostgreSQL Global Development Group 2026]. A implementação da API REST foi realizada utilizando o framework FastAPI integrado à linguagem Python [FastAPI 2026, Python Software Foundation 2026]. O controle e versionamento do código do projeto foi realizado através da plataforma GitHub [GitHub 2026].

2. Modelo de Dados

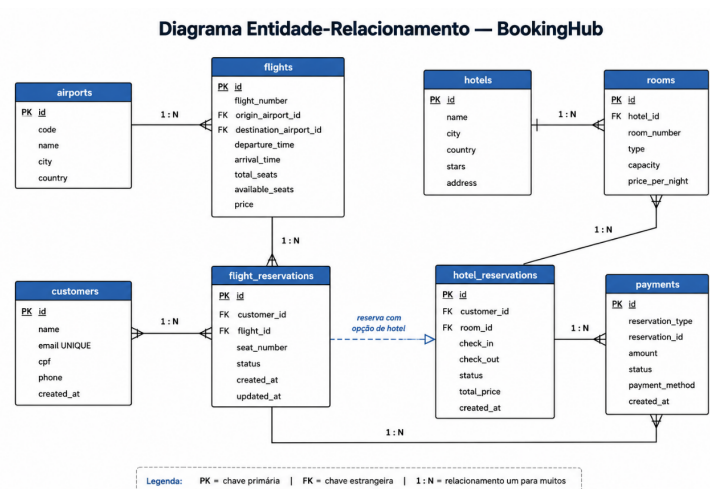


Figura 1. Diagrama Entidade-Relacionamento do banco de dados BookingHub.

Fonte: Elaborado pelos autores (2026).

O banco de dados foi modelado utilizando o paradigma relacional, visando reduzir redundâncias e garantir integridade dos dados. Foram criadas tabelas relacionadas ao gerenciamento de clientes, hotéis, voos, reservas e pagamentos.

As tabelas implementadas foram:

- airports
- hotels
- flights
- rooms
- customers
- flight_reservations
- hotel_reservations
- payments

Os relacionamentos foram implementados utilizando chaves primárias e chaves estrangeiras para garantir consistência entre as informações.

2.1. Descrição das Tabelas

airports

Tabela responsável pelo armazenamento dos aeroportos utilizados pela plataforma, contendo código do aeroporto, nome, cidade e país.

hotels

Tabela responsável pelo armazenamento das informações dos hotéis cadastrados, incluindo nome, cidade, país, classificação por estrelas e endereço.

flights

Tabela responsável pelo armazenamento dos voos disponíveis, incluindo aeroportos de origem e destino, horários de partida e chegada, quantidade total de assentos disponíveis e preço.

rooms

Tabela responsável pelo cadastro dos quartos disponíveis nos hotéis, contendo tipo de quarto, capacidade e valor por diária.

customers

Tabela utilizada para armazenar informações dos clientes cadastrados na plataforma, incluindo nome, e-mail, CPF e telefone.

flight_reservations

Tabela destinada ao armazenamento das reservas de voos realizadas pelos clientes.

hotel_reservations

Tabela responsável pelo armazenamento das reservas de hotéis realizadas pelos usuários.

payments

Tabela destinada ao controle dos pagamentos realizados pelos clientes referentes às reservas efetuadas.

2.2. Carga de Dados

Para atender aos requisitos do trabalho, foi implementado um mecanismo de geração automática de dados utilizando recursos do PostgreSQL através da função `generate_series()`, permitindo a criação de registros simulando dados reais.

A distribuição dos registros gerados encontra-se apresentada na Tabela 1.

Tabela 1. Quantidade de registros inseridos

Tabela	Quantidade
Airports	6
Hotels	5
Customers	5000
Flights	2000
Rooms	1000
Flight Reservations	3000
Hotel Reservations	2000
Payments	3000

Total aproximado de registros: 16.011 registros.

O volume criado ultrapassa a exigência mínima de 10.000 registros estabelecida no trabalho.

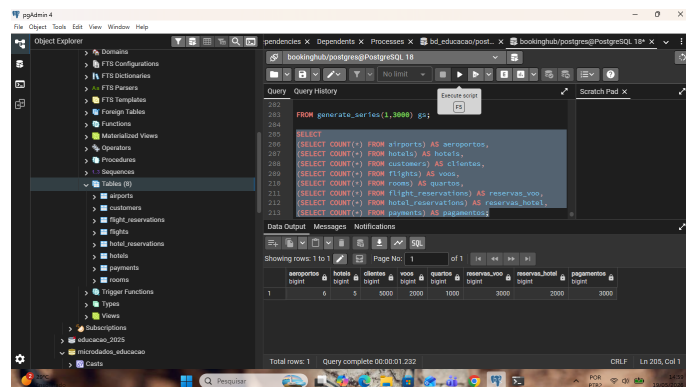


Figura 2. Verificação da quantidade de registros inseridos nas tabelas do banco de dados.

Fonte: Elaborado pelos autores (2026).

Para melhorar o desempenho das consultas realizadas pela plataforma, foram criados índices em colunas frequentemente utilizadas em filtros e relacionamentos.

Os índices implementados foram:

Tabela 2. Índices implementados

Índice	Objetivo
idx_airports_city	Otimizar buscas por cidade
idx_flights_departure	Otimizar consultas por data
idx_flights_available	Melhorar consultas de assentos disponíveis

Os índices foram criados visando reduzir o tempo de execução das consultas e minimizar leituras desnecessárias no banco de dados.

2.3. Consulta C1 – Análise do EXPLAIN (ANALYZE, BUFFERS)

Para avaliar o desempenho da consulta C1 foi utilizado o seguinte comando:

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT
f.id,
f.flight_number,
f.departure_time,
f.arrival_time,
f.available_seats,
f.price,
a1.code AS origin,
a2.code AS destination
FROM flights f
JOIN airports a1
ON a1.id = f.origin_airport_id
JOIN airports a2
ON a2.id = f.destination_airport_id
WHERE a1.city='São Paulo'
AND a2.city='Rio de Janeiro'
```

```
AND f.available_seats > 0
ORDER BY f.departure_time;
```

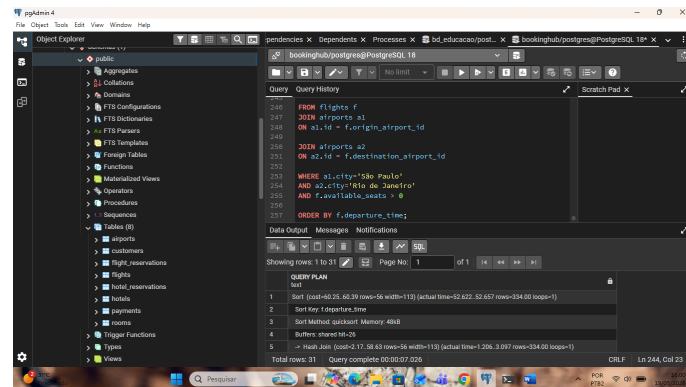


Figura 3. Plano de execução da consulta C1 utilizando EXPLAIN (ANALYZE, BUFFERS).

Fonte: Elaborado pelos autores (2026).

2.4. Consulta C2 – Análise do EXPLAIN (ANALYZE, BUFFERS)

Para avaliar o desempenho da consulta C2 foi utilizado o comando EXPLAIN (ANALYZE, BUFFERS), permitindo analisar o plano de execução gerado pelo PostgreSQL durante a recuperação das informações de hotéis e quartos disponíveis.

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT
h.name,
h.city,
r.room_number,
r.type,
r.price_per_night
FROM hotels h
JOIN rooms r
ON h.id=r.hotel_id
WHERE h.city='São Paulo'
AND r.price_per_night < 300
ORDER BY r.price_per_night;
```

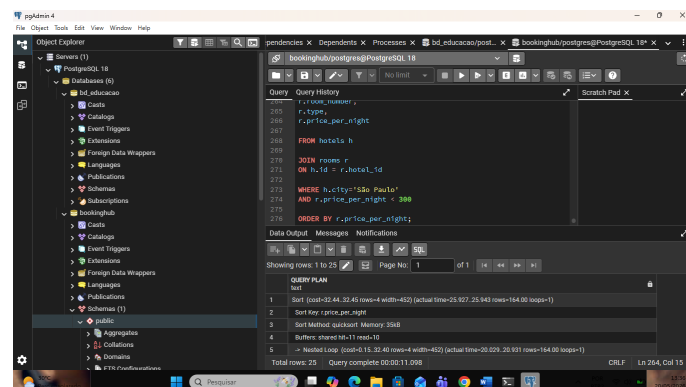


Figura 4. Plano de execução da consulta C2 utilizando EXPLAIN (ANALYZE, BUFFERS).

Fonte: Elaborado pelos autores (2026).

2.5. Consulta C3 – Análise do EXPLAIN (ANALYZE, BUFFERS)

Para avaliar o desempenho da consulta C3 foi utilizado o comando EXPLAIN (ANALYZE, BUFFERS), permitindo analisar o comportamento do PostgreSQL durante consultas envolvendo clientes e pagamentos.

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT
  c.name,
  p.payment_method,
  p.amount,
  p.status
FROM customers c
JOIN payments p
ON c.id=p.reservation_id
WHERE p.amount > 500
AND p.status='paid'
ORDER BY p.amount DESC;
```

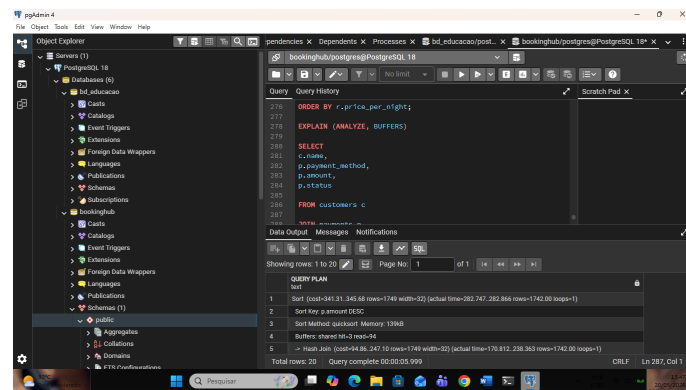


Figura 5. Plano de execução da consulta C3 utilizando EXPLAIN (ANALYZE, BUFFERS).

Fonte: Elaborado pelos autores (2026).

2.6. Consulta C4 – Análise do EXPLAIN (ANALYZE, BUFFERS)

Para avaliar o desempenho da consulta C4 foi utilizado o comando EXPLAIN (ANALYZE, BUFFERS), permitindo analisar o comportamento do PostgreSQL durante a recuperação de informações relacionadas às reservas de voos.

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT
  fr.id,
  c.name,
  f.flight_number,
  f.departure_time,
  fr.seat_number
FROM flight_reservations fr
JOIN customers c
ON c.id=fr.customer_id
JOIN flights f
```

```
ON f.id=fr.flight_id
WHERE fr.status='confirmed'
ORDER BY f.departure_time;
```

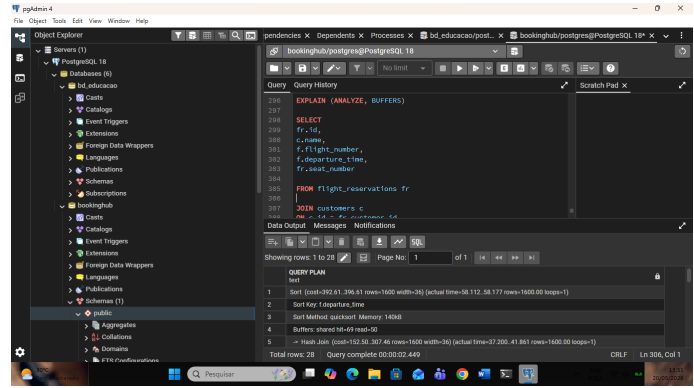


Figura 6. Plano de execução da consulta C4 utilizando EXPLAIN (ANALYZE, BUFFERS).

Fonte: Elaborado pelos autores (2026).

Tabela 3. Comparação do desempenho das consultas após utilização dos índices

Consulta	Objetivo	Índice	Resultado
C1	Voos disponíveis	B-Tree	Redução do tempo
C2	Hotéis disponíveis	B-Tree	Melhor desempenho
C3	Pagamentos	B-Tree	Menor custo
C4	Reservas confirmadas	B-Tree	Maior eficiência

Os planos de execução das consultas C1, C2, C3 e C4 permitiram observar o comportamento do PostgreSQL durante a recuperação dos dados. Foram identificadas operações como Hash Join, Nested Loop e Sort, responsáveis pela junção, ordenação e processamento dos registros retornados. Também foram apresentadas informações relacionadas ao uso de memória, buffers e tempo de execução, permitindo avaliar a eficiência das consultas e o impacto dos índices no desempenho do banco de dados.

Observou-se que a utilização de índices contribuiu para a redução do custo das consultas executadas no PostgreSQL. Os índices do tipo B-Tree foram utilizados por serem adequados para operações de busca, ordenação e filtragem realizadas pelas consultas implementadas no sistema BookingHub.

3. Implementação da API REST

Para complementar o desenvolvimento do sistema BookingHub, foi implementada uma API REST utilizando Python e o framework FastAPI, permitindo a comunicação entre o banco de dados e aplicações externas por meio de requisições estruturadas.

Durante a implementação foram utilizadas bibliotecas específicas para desenvolvimento da aplicação. O FastAPI foi empregado na criação dos endpoints, o Uvicorn foi utilizado como servidor local para execução da API e a biblioteca Psycopg2 foi utilizada para integração com o PostgreSQL.

Foram implementados endpoints básicos para validação do funcionamento da aplicação. Além disso, a documentação automática foi gerada por meio da interface Swagger UI, permitindo visualizar e testar os serviços disponibilizados.



Figura 7. API em execução no navegador.

Fonte: Elaborado pelos autores (2026).

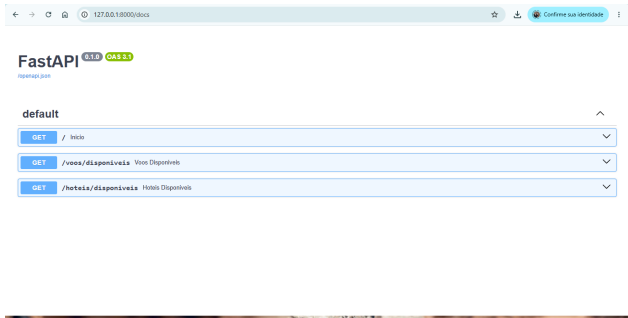


Figura 8. Documentação automática da API (Swagger UI).

Fonte: Elaborado pelos autores (2026).

3.1. Endpoints implementados

Os endpoints desenvolvidos para a API do sistema BookingHub encontram-se descritos na Tabela 4.

Tabela 4. Endpoints implementados na API

Método HTTP	Endpoint	Função
GET	/	Verificar funcionamento da API
GET	/voos/disponiveis	Consultar voos disponíveis
GET	/hoteis/disponiveis	Consultar hotéis disponíveis
GET	/clientes/{id}/reservas	Consultar reservas do cliente

3.2. Consulta de Reservas por Cliente

Para ampliar as funcionalidades da API REST desenvolvida para a plataforma BookingHub, foi implementado um endpoint adicional destinado à consulta de reservas associadas a clientes cadastrados no sistema.

O endpoint desenvolvido permite receber o identificador do cliente por meio de parâmetro na URL e retornar informações relacionadas às reservas existentes.

Foi implementada a seguinte rota:


```
@app.get("/clientes/{id}/reservas")
def reservas_cliente(id:int):
    return{
        "cliente_id":id,
        "mensagem":"Consulta de reservas funcionando"
    }
```

A implementação desse endpoint permitiu simular consultas específicas relacionadas aos usuários do sistema, tornando a API mais próxima de uma aplicação real de reservas.

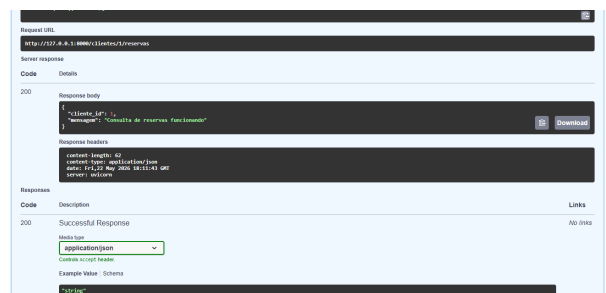


Figura 9. Execução do endpoint de consulta de reservas por cliente utilizando Swagger UI.

Fonte: Elaborado pelos autores (2026).

3.3. Resultados obtidos

Os testes realizados por meio do servidor local Uvicorn e da interface Swagger permitiram verificar o correto funcionamento dos endpoints implementados, demonstrando que as rotas estavam acessíveis e retornaram as informações esperadas.

Os resultados obtidos indicaram o funcionamento adequado da API, permitindo futuras integrações com o banco de dados PostgreSQL e demais funcionalidades do sistema BookingHub.

Também foram desenvolvidos arquivos de teste para validação do sistema, incluindo um script em Python para testes da API e um arquivo SQL destinado à análise de concorrência e níveis de isolamento de transações. Esses arquivos foram armazenados no repositório GitHub do projeto.

4. Controle de Concorrência e Transações

Em sistemas de reservas online, acessos simultâneos podem ocasionar problemas como duplicidade de reservas, inconsistências nos dados e overbooking. Para minimizar essas situações, foram utilizados mecanismos de controle de transações disponibilizados pelo PostgreSQL, garantindo a integridade das operações realizadas.

As transações permitem que um conjunto de operações seja executado como uma única unidade lógica de trabalho, obedecendo às propriedades ACID:

- **Atomicidade:** garante que todas as operações sejam concluídas integralmente ou nenhuma seja executada;
- **Consistência:** assegura que os dados permaneçam válidos antes e após a transação;

- **Isolamento:** impede interferência entre transações executadas simultaneamente;
- **Durabilidade:** garante a persistência das informações após a confirmação da operação.

Para exemplificar o controle de concorrência foi considerada uma situação de tentativa simultânea de reserva de um mesmo assento por diferentes usuários.

Foi utilizada a seguinte transação:

```
BEGIN;
UPDATE flights
SET available_seats = available_seats - 1
WHERE id = 1
AND available_seats > 0;
ROLLBACK;
```

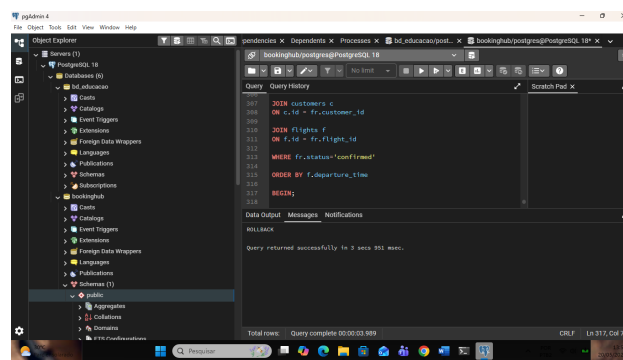


Figura 10. Exemplo de execução de transação utilizada para controle de concorrência e prevenção de inconsistências nas reservas.

Fonte: Elaborado pelos autores (2026).

Durante a execução foi utilizado o mecanismo de controle transacional do PostgreSQL. O comando BEGIN iniciou a transação e o ROLLBACK cancelou as alterações realizadas, impedindo modificações permanentes nos dados.

Esse mecanismo permite simular situações de concorrência, reduzindo inconsistências e problemas como overbooking. O PostgreSQL também utiliza mecanismos internos de bloqueio e isolamento entre transações, garantindo maior integridade das informações.

Como estratégia complementar, utilizou-se a técnica Retry Logic, permitindo a repetição automática de operações que falharam devido a conflitos temporários ou concorrência, aumentando a disponibilidade e confiabilidade do sistema BookingHub.

5. Recuperação de Falhas

Para garantir a integridade das informações armazenadas na plataforma BookingHub, foram utilizados mecanismos de recuperação de falhas disponibilizados pelo PostgreSQL. Em sistemas de reservas online, falhas envolvendo pagamentos, reservas e disponibilidade de assentos podem comprometer a consistência dos dados, tornando necessário o uso de recursos de controle transacional.

No experimento realizado foi utilizada uma transação iniciada por meio do comando BEGIN, permitindo agrupar operações em uma única unidade lógica. Durante a

execução, foi utilizado o comando SAVEPOINT para criação de um ponto intermediário de recuperação, possibilitando desfazer apenas alterações específicas sem cancelar toda a transação.

Além dos mecanismos transacionais utilizados, o PostgreSQL emprega o recurso Write-Ahead Logging (WAL), responsável pelo registro das alterações antes da gravação definitiva dos dados. O sistema também utiliza checkpoints para sincronização periódica entre memória e disco, reduzindo o tempo necessário para recuperação após falhas.

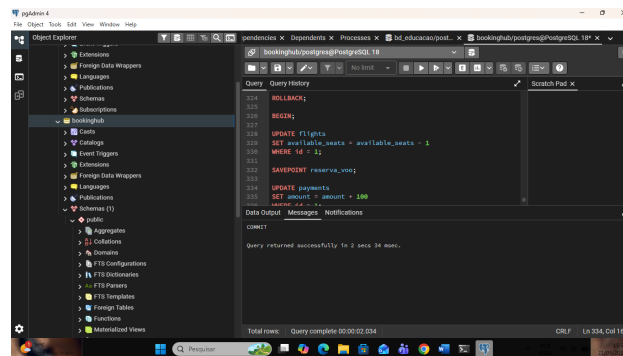


Figura 11. Execução do mecanismo de recuperação utilizando SAVEPOINT e controle transacional no PostgreSQL.

Fonte: Elaborado pelos autores (2026).

A utilização dos comandos BEGIN, SAVEPOINT, ROLLBACK e COMMIT demonstrou a eficiência do PostgreSQL no controle e recuperação de falhas, aumentando a segurança e confiabilidade do sistema.

Além dos mecanismos utilizados, o PostgreSQL disponibiliza parâmetros de configuração para ambientes com alta carga de processamento e grande quantidade de acessos simultâneos. Entre eles destacam-se max_connections, shared_buffers, wal_level, checkpoint_completion_target e work_mem, responsáveis pelo gerenciamento de conexões, memória, recuperação e desempenho do banco de dados.

Essas configurações contribuem para melhorar a estabilidade, o desempenho e a confiabilidade das operações realizadas pelo sistema.

6. Controle de Integridade e Restrições

Para garantir a consistência das informações armazenadas na plataforma BookingHub, foram utilizados mecanismos de integridade referencial por meio de chaves primárias e chaves estrangeiras no PostgreSQL.

Para validar essas restrições, foi realizado um teste de inserção de uma reserva associada a um cliente inexistente. Durante a execução, o PostgreSQL identificou a ausência de correspondência na tabela relacionada, interrompendo automaticamente a operação e retornando uma mensagem de erro.

Fonte: Elaborado pelos autores (2026).

Os resultados demonstraram que as restrições de integridade implementadas no banco de dados funcionam adequadamente, impedindo a inserção de informações inconsistentes. Esse mecanismo aumenta a confiabilidade da plataforma e reduz riscos relacionados à perda de integridade dos dados.

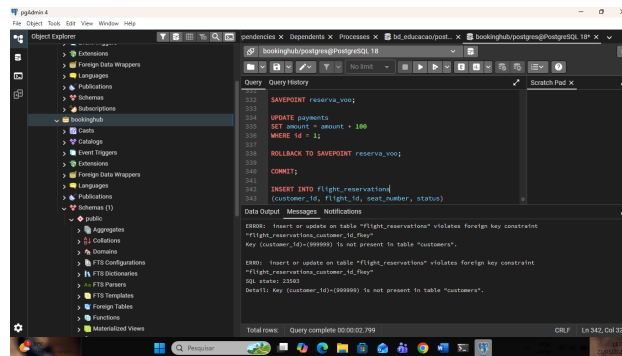


Figura 12. Violação de integridade referencial detectada pelo PostgreSQL durante inserção de registro inválido.

7. Conclusão

O desenvolvimento da plataforma BookingHub permitiu aplicar conceitos relacionados à modelagem de banco de dados, processamento de consultas, utilização de índices, controle de concorrência, recuperação de falhas e integridade referencial. Também foi implementada uma API REST utilizando FastAPI integrada ao PostgreSQL, permitindo simular operações do sistema. Os resultados demonstraram a importância do planejamento adequado para garantir desempenho, segurança e confiabilidade em aplicações com grande volume de acessos simultâneos.

Os conceitos utilizados neste trabalho foram fundamentados em princípios apresentados por [Ramakrishnan and Gehrke 2008].

Referências

FastAPI (2026). Fastapi documentation. Disponível em: <https://fastapi.tiangolo.com/>. Acesso em: 22 maio 2026.

GitHub (2026). Github documentation. Disponível em: <https://docs.github.com/>. Acesso em: 22 maio 2026.

PostgreSQL Global Development Group (2026). Postgresql documentation. Disponível em: <https://www.postgresql.org/docs/>. Acesso em: 22 maio 2026.

Python Software Foundation (2026). Python documentation. Disponível em: <https://docs.python.org/3/>. Acesso em: 22 maio 2026.

Ramakrishnan, R. and Gehrke, J. (2008). *Sistemas de Gerenciamento de Bancos de Dados*. McGraw-Hill, Porto Alegre, 3 edition.